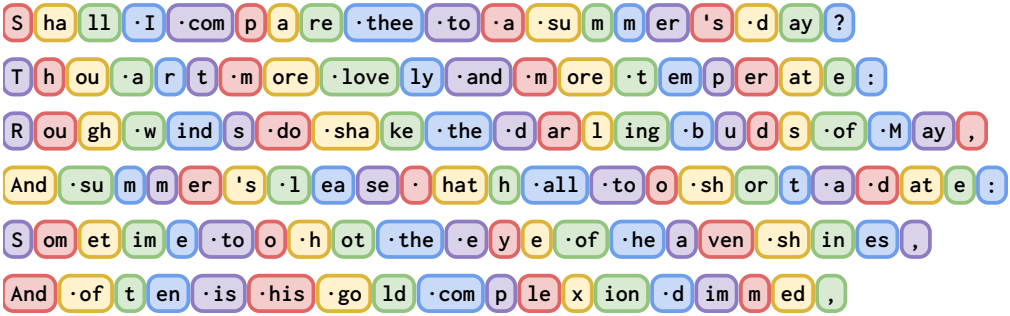


UNIT 01 · CHAPTER 5

BPE Tokenization

CSE 10124 · Building ChatGPT
William Theisen · University of Notre Dame

Subword Vocabularies

The last chapter ended with three ways to split text: words (a huge vocabulary that still misses words), characters (texts several times longer) and irishQwen's **512** subword tokens in between. Here is the first line of Sonnet 18 all three ways:

Word level — 9 tokens

`Shall` `I` `compare` `thee` `to` `a` `summer's` `day` `?`

Character level — 39 tokens

`S` `h` `a` `l` `l` `I` `c` `o` `m` `p` `a` `r` `e` `t` `h` `e` `e` `t` `o` `a` `s` `u` `m` `m` `e` `r` `'` `s` `d` `a` `y` `?`

Subword level (the class tokenizer) — 19 tokens

`S` `ha` `ll` `·I` `·com` `p` `a` `re` `·thee` `·to` `·a` `·su` `m` `m` `er` `'s` `·d` `ay` `?`

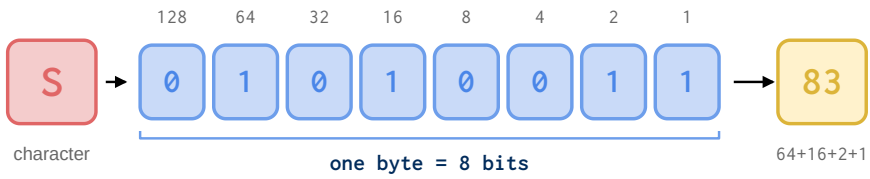
Common words get a token to themselves (" thee"), rarer ones are built out of pieces ("com" + "p" + "a" + "re"), and the space in front of a word travels with it (drawn as a dot).

Nobody chose those pieces. They were learned from Shakespeare by counting which bits of text turn up next to each other most often, with an algorithm called **Byte-Pair Encoding**, and **the list of pieces it learns is the tokenizer**. Before we can count anything, though, we need the smallest pieces to start from.

Bytes

The Shakespeare file only uses **91** different characters, but a chatbot's prompt can contain anything: accented letters, Chinese, emoji, code. Unicode, the standard list of every character computers know about, has well over **150,000** of them, which is a big vocabulary before we've merged a thing.

Underneath, a computer stores all of them the same way, as bytes. A **bit** is a single **0** or **1**, and a **byte** is **8** bits, so a byte holds one of $2^8 = 256$ values, **0** to **255**. The letter S, for instance, is stored as the byte **83**:



If our tokens start from bytes instead of characters, **the starting vocabulary is exactly 256 tokens**, and there is no text in any language that can't be written with them.

ASCII and UTF-8

Which byte stands for which character is decided by an **encoding**. The oldest one still in everyday use is **ASCII** (1963), which gives **128** characters a number each: **32** is a space, **65** to **90** are A to Z and **97** to **122** are a to z. That covers English and not much else.

UTF-8, the encoding almost every web page now uses, keeps all of ASCII as single bytes and writes every other character with two, three or four. Python shows us the bytes behind any string with `.encode`:

```
1  text = "hello there 金"
2  data = text.encode("utf-8")
3  print(len(text), "characters,", len(data), "bytes")
4  print(list(data))
5  print(list("Shall".encode("utf-8")))
```

OUTPUT

```
13 characters, 15 bytes
[104, 101, 108, 108, 111, 32, 116, 104, 101, 114, 101, 32, 233, 135
[83, 104, 97, 108, 108]
```

The first twelve characters are one byte each, but 金 (Chinese for gold) takes three. Shakespeare is pure ASCII, so in our corpus one character is one byte. irishQwen's training script also strips out Project Gutenberg's notices, the indentation and extra blank lines, leaving **4,875,049** characters, and so **4,875,049** bytes.

Counting Pairs

On its own, though, a byte-level tokenizer is just a character-level tokenizer again, with the same long sequences. What we want is a way to glue bytes that often appear together into bigger tokens, and to find out which bytes belong together, we count them. Here is all of Sonnet 18 as bytes (**621** of them), with every pair of neighboring bytes counted:

```
from collections import Counter
text = open("shakespeare.txt").read()
start = text.index("Shall I compare")
end = text.index("life to thee.", start) + 13
lines = text[start:end].split("\n")
sonnet = "\n".join(line.strip() for line in lines)
ids = list(sonnet.encode("utf-8"))
counts = Counter(zip(ids, ids[1:]))
print(len(ids), "bytes")
print(counts.most_common(4))
```

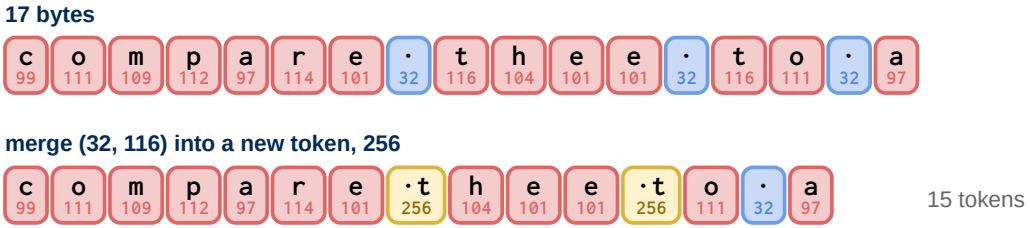
OUTPUT

```
621 bytes
[((32, 116), 18), ((101, 32), 16), ((116, 104), 14), ((115, 32), 14
```

The most common pair is `(32, 116)`, a space followed by a t, which shows up **18** times (the start of " thee", " to", " the", " thou" and friends). Next come "e " (**16** times), "th" (**14**) and "s " (**14**). None of this would surprise anyone who reads English, but **the computer found it just by counting**.

Merging Pairs

Since " t" turns up so often, we make it a token. Bytes use the ids `0` to `255`, so it gets the next id, `256`, and every `(32, 116)` becomes a single `256`:



In code:

```python
def merge(ids, pair, new_id):
    out, i = [], 0
    while i < len(ids):
        if tuple(ids[i:i + 2]) == pair:
            out.append(new_id)
            i += 2
        else:
            out.append(ids[i])
            i += 1
    return out

ids = merge(ids, (32, 116), 256)
print(len(ids), "tokens")
print(ids[:18])
```

OUTPUT

```
603 tokens
[83, 104, 97, 108, 108, 32, 73, 32, 99, 111, 109, 112, 97, 114, 101
```

The sonnet went from `621` tokens to `603`, and the vocabulary from `256` tokens to `257`. Nothing was lost (`256` always turns back into `32` and `116`), so why stop at one?

Byte-Pair Encoding

Then we repeat: count the pairs again, merge the most common into the next id, and so on. The next six merges on Sonnet 18 (new id, pair, its bytes, how often it appeared, new length):

```
vocab = {i: bytes([i]) for i in range(256)}
vocab[256] = b" t"
for new_id in range(257, 263):
    counts = Counter(zip(ids, ids[1:]))
    pair, n = counts.most_common(1)[0]
    ids = merge(ids, pair, new_id)
    vocab[new_id] = vocab[pair[0]] + vocab[pair[1]]
    print(new_id, pair, vocab[new_id], n, len(ids))
```

OUTPUT

```
257 (115, 32) b's ' 12 591
258 (256, 104) b' th' 11 580
259 (109, 101) b'me' 11 569
260 (32, 115) b' s' 10 559
261 (44, 10) b',\n' 9 550
262 (104, 97) b'ha' 8 542
```

Merge **258** joins **256** (" t") with an h. Merged tokens can be merged again. This loop is called **byte-pair encoding** (BPE). It began as a data compression trick (Philip Gage, 1994), was borrowed for machine translation in 2016, and since GPT-2 (2019) a version that starts from bytes has built the tokenizers of most large language models.

Each merge adds a token to the vocabulary and shortens the text. After seven merges the sonnet is **542** tokens long, and **the list of merges, in the order we made them, is the tokenizer**.

Merges Across Words

The second most common pair in Sonnet 18 was "e ", the end of one word glued to the gap before the next. BPE only counts, it has no idea where words begin and end, and on more text that gets silly. Here are some tokens BPE learns from the first **300,000** characters of that training text:

merge	new token	times the pair appeared
1	"e "	7,503
72	"e, "	485
102	", and "	335
253	"PAROLLES. "	141
402	"to the "	83
499	"e that "	64

Each spends a token on a word plus whatever came after it. "to the " is a fine phrase but a strange token, and "PAROLLES. " (a character in *All's Well That Ends Well*) is a token only because the script puts his name, a period and a space in front of each of his lines.

The same thing happens in modern text. Left alone, BPE can learn "tonight?" as a token that has nothing to do with "tonight", and the model then has to learn what each one means separately. **We want merges to stay inside words**.

Regular Expressions

To keep merges inside words, we first need a way to say what a word looks like. The standard tool for describing patterns in text is a **regular expression** (regex for short), a small language of its own. `\b` matches the edge of a word, `|` means "or", `+` means "one or more of the thing before", and most other characters just match themselves. Python's `re` module finds every match of a pattern in a string:

```python
import re
text = open("shakespeare.txt").read()
words = ["thou", "thee", "thy", "tonight", "homework"]
for word in words:
    found = re.findall(r"\b" + word + r"\b", text)
    print(word, len(found))
```

OUTPUT

```
thou 4594
thee 3162
thy 3634
tonight 47
homework 0
```

Shakespeare wrote "thou" **4,594** times, "thee" **3,162** times and "homework" never ("tonight" turns up **47** times, though he more often wrote it "to-night").

Regex is famously hard to read, and **almost nobody writes one from memory**. A cheat sheet (or a chatbot) is the normal way to build one. All we need from it here is a pattern for chopping text into words.

Pre-Tokenization

GPT-4's tokenizer splits text with a regex before BPE ever sees it. The pattern looks alarming, but it only breaks text into a few kinds of chunk: a word with at most one space or punctuation mark in front of it, runs of up to three digits, runs of punctuation, and whitespace. Here it is on Hamlet:

```python
import regex
GPT4 = (r"'(?i:[sdmt]|ll|ve|re)|[^\r\n\p{L}\p{N}]?+"
        r"\p{L}+|\p{N}{1,3}| ?[^\s\p{L}\p{N}]++[\r\n]*"
        r"|\s*[\r\n]|\s+(?!\S)|\s+")
line = "To be, or not to be- that is the question:"
print(regex.findall(GPT4, line))
```

OUTPUT

```
['To', ' be', ',', ' or', ' not', ' to', ' be', '-', ' that', ' is'
```

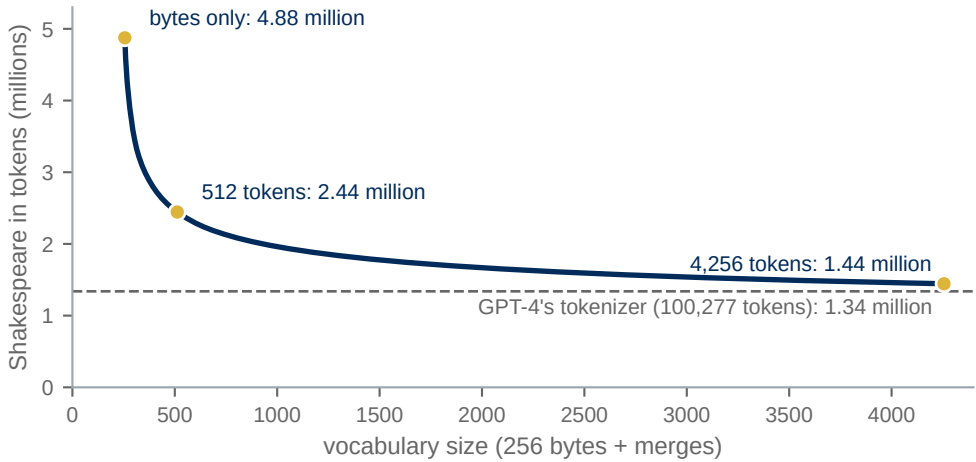
(The pattern needs Python's `regex` module rather than `re`, since `re` doesn't understand `\p{L}`, "any letter in any language".) Those **13** chunks are the **pre-tokenization**:

`To` `·be` `,` `·or` `·not` `·to` `·be` `-` `·that` `·is` `·the` `·question` `:`

BPE then counts pairs only inside a chunk, never across two. A space stays attached to the front of the word after it (" be", " question"), and **the comma after "be" can never be glued onto it**, so "e " and "to the " are off the table.

Training on Shakespeare

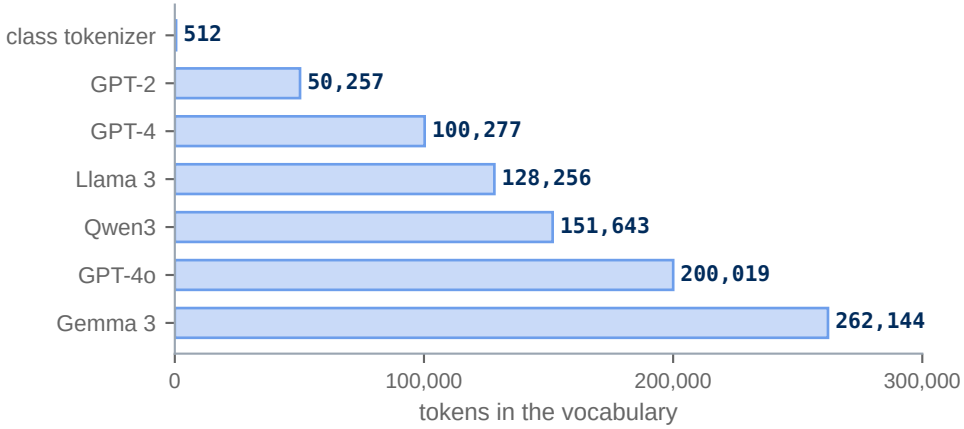
With pre-tokenization in place we can train on the whole corpus. Counting pairs inside every chunk of all **4,875,049** bytes, the first merges are " t" (**99,544** times), "he", " a", "ou", " s" and "in", and merge **13** joins " t" and "he" into " the". Further down the list come whole words: " thou" is merge **105**, " thee" merge **183**, " Romeo" merge **2,138** and " Hamlet" merge **2,554**.



Every merge shortens the text, quickly at first and then more and more slowly. The first **256** merges (a vocabulary of **512**) take Shakespeare from **4.88** million tokens to **2.44** million, half its length, but the next **3,744** only get it down to **1.44** million. **The pairs left to merge are rare**, so merging them doesn't save much.

Vocabulary Size

So how many merges should we make? It's a choice, and it sets the **vocabulary size**. More merges mean shorter sequences, which makes everything that reads them cheaper, but every token in the vocabulary needs its own probability at the model's output (and its own row of numbers at the input, as the next chapter shows). Rare tokens also get seen rarely, so a huge vocabulary spreads the training text thin.



The class tokenizer stops at `512`, since irishQwen is small and only ever reads Shakespeare. Production models go much bigger, and the trend is up: GPT-2 had `50,257` tokens, GPT-4 has `100,277` and Gemma 3 `262,144`. A 2024 study of the question (Tao et al.) was titled, fairly plainly, *Larger Models Deserve Larger Vocabularies*.

A Tokenizer of Our Own

Training a tokenizer puts the last few pages together. Split the text into chunks with the GPT-4 pattern, turn each chunk into bytes, and run the merge loop over all the chunks at once. Here it is on the first `20,000` characters of the Sonnets (about the first `30`), with `100` merges:

```
text = open("shakespeare.txt").read()
start = text.index("THE SONNETS")
lines = text[start:start + 20000].split("\n")
sonnets = "\n".join(line.strip() for line in lines)
chunks = [list(c.encode("utf-8"))
          for c in regex.findall(GPT4, sonnets)]
print(sum(len(c) for c in chunks), "bytes")
merges = {}
for new_id in range(256, 356):
    counts = Counter()
    for c in chunks:
        counts.update(zip(c, c[1:]))
    pair = counts.most_common(1)[0][0]
    merges[pair] = new_id
    chunks = [merge(c, pair, new_id) for c in chunks]
print(sum(len(c) for c in chunks), "tokens")
```

OUTPUT

```
18380 bytes
10143 tokens
```

(`merge` and `Counter` are from earlier pages. Stripping the indentation from every line leaves `18,380` bytes.) `100` merges cut the text nearly in half.

The tokenizer itself is just the dictionary `merges`: which pair became which id, in the order they were learned.

Encoding

First we build a `vocab`, the bytes behind every id. **Encoding** new text means replaying the merges: split it into chunks, turn each chunk into bytes, and apply every merge in the order it was learned (Python dictionaries remember their order). Hamlet's line isn't in the Sonnets, so it's a fair test:

```
vocab = {i: bytes([i]) for i in range(256)}
for (a, b), new_id in merges.items():
    vocab[new_id] = vocab[a] + vocab[b]
print([vocab[i] for i in range(346, 356)])

def encode(text):
    ids = []
    for chunk in regex.findall(GPT4, text):
        c = list(chunk.encode("utf-8"))
        for pair, new_id in merges.items():
            c = merge(c, pair, new_id)
        ids += c
    return ids

ids = encode(line)
print(ids)
print([vocab[i].decode("utf-8") for i in ids])
```

OUTPUT

```
[b'ke', b' wh', b'.\n', b'ine', b' u', b'\n\n\n', b'ge', b' love',
[84, 111, 332, 44, 271, 114, 340, 287, 332, 45, 334, 32, 290, 268,
['T', 'o', ' be', ',', ' o', 'r', ' not', ' to', ' be', '-', ' that
```

The last ten tokens learned include " love" and the end of "beauty". On Hamlet, common pieces come out whole (" be", " not", " that", " the"), and "question", which `100` merges never saw enough of, is spelled out from small pieces.

Decoding

Decoding goes the other way, gluing together the bytes behind each id. Here it is on Hamlet, and on something the Sonnets definitely don't contain:

```
def decode(ids):
    return b"".join(vocab[i] for i in ids).decode("utf-8")

print(decode(ids))
new = encode("Help me with my homework 金")
print(new)
print([vocab[i] for i in new])
print(decode(new))
```

OUTPUT

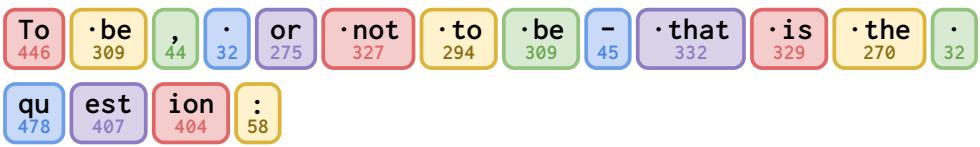
```
To be, or not to be- that is the question:
[72, 101, 108, 112, 265, 101, 324, 317, 279, 111, 298, 119, 278, 10
[b'H', b'e', b'l', b'p', b' m', b'e', b' with', b' my', b' h', b'o'
Help me with my homework 金
```

Decoding gives back exactly the text we started with. "homework" was never seen, so it comes out in pieces (" h", "o", "me", "w", "or", "k"), and 金 is just its three bytes, **233**, **135** and **145**, because no merge ever involved them. **Nothing is ever unknown**, the worst case is one token per byte.

The Class Tokenizer

irishQwen's tokenizer (`Regex_Tokenizer` in `irishQwen/tokenizer.py`) is the same algorithm with one addition. It reserves three **special tokens**: <|pad|> (**256**), <|sos|> (**257**, start of sequence) and <|eos|> (**258**, end of sequence), the same three the last chapter met, which are never split. Its merges start at **259**, and with **253** of them, learned from Shakespeare, the vocabulary comes to exactly **512**.

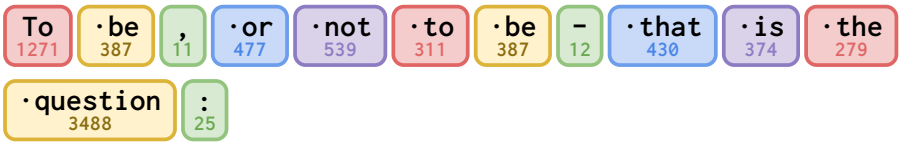
Its first eleven merges are the same as the ones we found on all of Shakespeare (" t", "he", " a", "ou" and so on), and Hamlet's line comes out as **17** tokens between its <|sos|> and <|eos|>:



With **512** tokens it turns the **4,875,049** characters of the training text into **2,526,264** tokens, about **1.9** characters per token. The training script wraps the text in <|sos|> and <|eos|> first, so the sequence irishQwen trains on is **2,526,266** tokens long, and **every prediction it makes is a probability for each of those 512 tokens**.

GPT-4's Tokenizer

GPT-4's tokenizer, `cl100k_base`, is the same idea at a much larger scale: the same pre-tokenization pattern (it's where ours came from), byte-level BPE, and **`100,277`** tokens learned from far more than Shakespeare. OpenAI publishes it in the Python library `tiktoken`. In it, every word of Hamlet's line is a single token:



It encodes irishQwen's training text in **`1,338,926`** tokens, about **`3.6`** characters per token (most words are a single token). GPT-4o and OpenAI's later models use `o200k_base`, with **`200,019`** tokens, which gets the same text down to **`1,317,273`**. **Doubling the vocabulary saved less than 2%** on Shakespeare, which is the flat end of the curve on the Training on Shakespeare page.

Token IDs and Meaning

So far a token id has been just a number. In GPT-4's tokenizer " thou" is **34223**, " thee" is **40344** and " thy" is **26236**, three forms of the same word with nothing in common as numbers. And "thou" at the start of a line, with no space in front, isn't **34223** at all, it's two tokens, "th" (**339**) and "ou" (**283**):



The ids are labels, like jersey numbers. Token **34223** isn't "close" to token **34224** in any way that matters, and nothing about **40344** says it's a form of **34223**. Models cope (ChatGPT, asked for the opposite of "drty", answers "clean", even though "drty" is two tokens it rarely sees), but **every connection like that has to be learned**.

Our own tokenizer's ids are no better. In it " be" is **332** and " to" is **287** only because of the order its merges happened to be learned in.

Token IDs as Input

What a language model actually receives is a tensor of token ids, one per token. Here is Hamlet's line from our own tokenizer, ready for PyTorch:

```
import torch
ids = torch.tensor(encode(line))
print(ids)
print(ids.shape, ids.dtype)
```

OUTPUT

```
tensor([ 84, 111, 332,  44, 271, 114, 340, 287, 332,  45, 334,  32,
torch.Size([22]) torch.int64
```

irishQwen gets the same thing on a larger scale: all of Shakespeare as **2,526,266** ids from the class tokenizer, cut into windows of **128** tokens. Each window is a tensor of shape `[128]` holding whole numbers between **0** and **511**, and nothing more.

Before a network can do anything useful with them, **each id has to become something that can carry meaning**: a list of numbers, where tokens that are used in similar ways end up close together. Those lists are **text embeddings**.